第 04 章

LCEL出题、结构化输出与质量门禁

沿着一份资料到题库的真实路径，读懂固定 Chain 的编排与可追溯校验。

这一章完成后，你能够

- 区分固定 Chain、Celery 任务和辅导 Agent 的职责
- 读懂 Prompt、模型与 Pydantic Parser 的输入输出
- 解释字符预算、引用验证、去重和题目状态
- 运行离线实验，亲自观察结构合法但引文错误的候选题

学习前提

完成资料解析与切块章节；理解 Python 列表、字典、类型标注和 UUID；能够使用项目已锁定依赖的 Python 环境运行脚本

代码基准：72dea46 + 当前已完成的学习准备。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

4.1 业务场景：资料解析完成以后，题目从哪里来? 3

4.2 名词解析：LCEL、Runnable 和 Parser 如何配合? 4

4.3 材料输入：字符预算为什么会影响覆盖范围? 5

4.4 第一条 Chain：先把原文变成知识点候选 6

4.5 第二条 Chain：题型结构为什么必须先确定? 7

4.6 质量门禁：结构合法的题，仍可能被拒绝 9

4.7 保存与界面：为什么会出现 partial 和待审核? 11

4.8 完整离线实验：一次解析，两种质量结果 13

4.9 常见错误、练习与参考思路 16

4.10 综合练习与参考思路 16

4.11 源码阅读清单与下一步 17

4.1 业务场景：资料解析完成以后，题目从哪里来？

上传资料只解决“系统能读取内容”。学习者还需要可作答的题目、明确的标准答案以及能回到原文的依据。本章以 TCP 三次握手短文作为独立教学输入，追踪知识点抽取、题目生成、程序校验和入库这条路径。项目页面中的真实资料另有名称，图注会区分页面观察与教学短文。

一次生成请求包含目标题数、允许题型、难度范围和语言。应用服务先确认资料已经 ready、知识库有效且没有冲突任务，再创建 pending 作业并投递 Celery。后台 Worker 执行固定步骤；HTTP 请求无需一直等待模型完成，也不会让模型自行决定是否把结果写入数据库。



图 4-1 通过校验的主路径。箭头表示程序顺序；拒绝分支见 4.6 节质量门禁。



图 4-2 真实题库页面。截图对应“聚时”知识库，观察题型、题目状态和启用入口；页面数量仅表示截图时的业务数据。

当前真实项目页面 / 现有业务数据 / 2026-10-07。题库审核状态及原文引用数量

本章观察任务

先找出待审核题与已启用题，再思考：为什么模型返回一道题以后，还需要校验和人工启用？把“生成完成”与“能够开始练习”分开理解。

4.2 名词解析：LCEL、Runnable 和 Parser 如何配合?

LCEL 是 LangChain Expression Language，即用表达式组合可执行组件的方式。这里的竖线符号 | 表示把上一步输出交给下一步：Prompt 生成消息，模型返回内容，Parser 把内容转换成业务模型。三个组件组成可调用的 Runnable 链，程序通过 invoke 或 ainvoke 执行它。

名词 / 函数	直观含义	本项目用途
ChatPromptTemplate	含变量的聊天消息模板	分别规定抽取与出题的角色、原文和约束
partial	提前固定部分模板变量	注入格式说明，调用时不用再次传入

名词 / 函数	直观含义	本项目用途
Runnable	可统一调用的执行组件	以 dict 为输入，以知识点或题目批次为输出
PydanticOutputParser	把模型文本解析成类型对象	执行字段长度、取值和题型结构校验
with_structured_output	配置模型的结构化返回方式	在配置选择非 parser 路径时使用
Celery Task	异步后台作业	负责阶段推进、状态记录和任务级重试

表 4-1 把术语落实到本章的真实作用。

不要把“链中使用了模型”直接理解为 Agent。当前出题顺序由 tasks.py 写定，模型只能产生知识点和题目候选；它没有工具选择循环。辅导 Agent 才会在约束范围内调用资料搜索、进度和错题工具。Celery、LCEL 和 Agent 是不同层面的机制，可以同时存在于一个项目。

阅读类型标注 `Runnable[dict[str, Any], QuestionBatch]` 时，可以把方括号两侧当成接口合同：调用方传入字典，成功以后收到 QuestionBatch 对象。这样下游读取 items，不必猜模型是否返回 Markdown、列表或解释文字。但成功解析只证明结构满足要求，还不能证明答案有充分依据。

4.3 材料输入：字符预算为什么会影响覆盖范围?

Worker 从 PostgreSQL 按 chunk_index 读取当前资料片段，再调用 build_generation_context。每段保留真实 chunk_id、标题路径和起始页码，让模型输出来源 ID 时有可选对象。这里没有用户检索问题，也没有调用 Milvus 召回片段；它采用材料顺序拼接，输入范围主要由字符预算决定。

源码节选：材料上下文拼接

定位：apps/api/study_agent/question_generation/tasks.py，第 96-107 行。

```python
def build_generation_context(chunks: list[DocumentChunk], max_chars: int) -> str:
    """Join chunks within the existing character budget, preserving legacy behavior."""
    sections: list[str] = []
    used = 0
    for chunk in chunks:
        heading = " / ".join(chunk.heading_path) or "无标题"
        section = f"[chunk_id={chunk.id}; 标题={heading}; 页码={chunk.page_start}]\n{chunk.content}"
        if sections and used + len(section) > max_chars:
            break
        sections.append(section)
        used += len(section)
    return "\n\n".join(sections)
```

chunks 是前面按 chunk_index 排序得到的 DocumentChunk 列表；max_chars 是字符预算。

返回 str，每一节包含来源头部和正文；标题路径用斜线连接，缺标题时显示“无标题”。

sections 非空以后才执行超额中止，因此首个片段即使很长也会加入。

输入 / 边界	实际行为	学习时的判断
默认 max_chars = 24000	配置限制累计 section 长度	字符数不是精确 Token 数
第一个 section 超额	仍然放入上下文	该参数不是严格硬上限
后续 section 超额	立即 break，不再看后面的片段	后章内容可能未被模型看到
段间空行	最终 join 加入两个换行	空行长度未计入 used
页码为 None	来源头部仍包含该值	不能据此虚构 PDF 页码

表 4-2 根据真实代码解释预算，避免把配置名读成更强的保证。

假设有五章材料，前两章已经消耗预算，后面第三到第五章就不会进入本次输入。即使用户请求很多题，也不意味着模型能够覆盖整份长资料。完整覆盖需要章节分组、分批生成和覆盖审计等额外设计；当前实现尚未具备这些保证，不能在 PDF 中写成已完成的功能。

为什么保存 chunk_id

知识点与题目引用应指向数据库中真实片段。页码和标题帮助人理解，chunk_id 帮助程序建立关系。UUID 结构合法、引用存在、引文匹配，是三个不同检查。

4.4 第一条 Chain：先把原文变成知识点候选

知识点是组织题目的中间层。例如从“TCP 通过三次握手建立连接”抽取“三次握手”，并赋予稳定键 tcp-handshake。名称用于展示，canonical_key 用于匹配与去重，source_chunk_ids 记录来源；importance 和 difficulty 是候选属性，其数值范围被 Schema 限制，不能直接当成已经验证的学习效果。

源码节选：知识点 Chain 的模板、模型与解析器

定位：apps/api/study_agent/question_generation/chains.py，第 15-40 行。

```python
def create_knowledge_point_chain(
    settings: Settings,
) -> Runnable[dict[str, Any], KnowledgePointBatch]:
    parser = PydanticOutputParser(pydantic_object=KnowledgePointBatch)
    prompt = ChatPromptTemplate.from_messages(
        [
            (
                "system",
                "你是职业学习资料分析器。只根据提供的原文片段提取可考查知识点。"
                "不得编造片段 ID；canonical_key 应稳定、简短、可用于去重。",
            ),
            (
                "human",
                "语言：{language}\n原文片段（每段含真实 chunk_id）：\n{context}\n\n"
                "输出要求：\n{format_instructions}",
            ),
```

代码续

```
        ]
    ).partial(format_instructions=parser.get_format_instructions())
    model = get_chat_model(settings, purpose="question_generation")
    if settings.llm_structured_output_method == "parser":
        chain = prompt | model | parser
    else:
        chain = prompt | model.with_structured_output(
            KnowledgePointBatch, method=settings.llm_structured_output_method
        )
    return cast(Runnable[dict[str, Any], KnowledgePointBatch], chain)
```

Settings 是依赖配置；KnowledgePointBatch 规定输出批次，每批至少一项。

调用字典包含 language 与 context；format_instructions 已经由 partial 固定。

默认 parser 路径返回 prompt | model | parser；其他路径使用模型结构化接口。

字段 / 参数	限制或含义	TCP 示例
canonical_key	2-300 字符；表达稳定知识点	tcp-handshake
description	5-2000 字符；解释可考内容	理解 TCP 建立连接的过程
importance / difficulty	0-1 / 1-5	0.9 / 2，只是模型候选属性
source_chunk_ids	1-5 个 UUID	指向提供给模型的原文片段
KnowledgePointBatch.items	1-30 个候选知识点	一次抽取的结构化集合

表 4-3 知识点模型不是任意字典。

任务随后把模型给出的来源 ID 与真实片段集合比较：删去不存在的 ID，只保留至少含一个有效来源的知识点。如果全部候选失去来源，任务报错。保存时会规范化 canonical_key 并查询知识库已有知识点，因此同一知识点可以关联多个来源，而不是每次都创建一个新名称。

这个阶段的来源过滤还没有验证知识点描述能否被原文推出。它约束 ID 的真实性，但不替代内容审核。阅读时应逐层记录“这一行代码证明了什么”：格式、范围、关系或语义。只有清楚边界，才能在后续扩展时找到真正欠缺的环节。

4.5 第二条 Chain：题型结构为什么必须先确定?

第二次调用把已校验来源的知识点序列化成 JSON，与相同材料上下文、题数和难度范围一起交给出题模型。serialize_knowledge_points 使用 model_dump(mode="json") 转换 UUID 等对象，再用 ensure_ascii=False 保留中文。这是输入数据准备，不是另一轮模型推理。



图 4-3 真实生成配置：指定目标题数、题型和难度范围。这里只打开配置窗口，未提交新任务；当前 6 道是界面输入值，不是上一轮 10 道任务的结果。

当前真实项目页面 / 现有业务数据 / 2026-10-07。题型和目标题数配置；只打开界面，未提交生成

源码节选：题型结构验证

定位：apps/api/study_agent/question_generation/schemas.py，第 38-61 行。

```
    @model_validator(mode="after")
    def validate_type_shape(self) -> "GeneratedQuestionDraft":
        if self.question_type == "single_choice":
            if self.options is None or len(self.options) != 4:
                raise ValueError("单选题必须包含 A、B、C、D 四个选项")
            if {option.key for option in self.options} != {"A", "B", "C", "D"}:
                raise ValueError("单选题选项键必须为 A、B、C、D")
            if len(self.correct_answers) != 1 or self.correct_answers[0] not in {
                "A",
                "B",
                "C",
                "D",
            }:
                raise ValueError("单选题必须且只能有一个正确选项")
        elif self.question_type == "true_false":
```

代码续

```
            self.options = None
            if len(self.correct_answers) != 1 or self.correct_answers[0] not in {
                "正确",
                "错误",
            }:
                raise ValueError("判断题答案必须为正确或错误")
        else:
            self.options = None
        return self
```

该 after validator 在基本字段解析之后执行；self 是已经构造中的题目候选。

单选要求 A-D 四个键、一个合法答案；判断只允许“正确”或“错误”。

判断和填空的 options 会被设置为 None。它没有检查选项文本是否互异，该检查在下一层。

题型	返回结构	示例与注意事项
single_choice	四个 QuestionOption；correct_answers 只有一个键	答案 ["C"]；选项文字互异由质量函数检查
fill_blank	options 为 None；允许多个答案别名	["三次", "3次"] 表示可接受表达，不是多个空
true_false	options 为 None；答案仅“正确”或“错误”	生产生成接口不接受 true 或 1 作为标准答案
共有字段	知识点键、题干、解析、难度、片段与引文	source_chunk_ids 与 source_quotes 应一一对应

表 4-4 生成结构与答题时的用户输入规则并不相同。

Pydantic 的 Field 控制长度和数值范围，Literal 限定题型和选项键，model_validator 处理跨字段关系。比如只有一个选项的单选题会在构造候选时触发 ValidationError，根本不会进入引用质量函数。检查错误位置时，要先判断是在文本解析、结构校验还是业务校验失败。

与课程示例的对应

飞书课程先定义输出类，再说明如何取得类型对象。本章延续这种讲法，但固定 Chain 直接返回 QuestionBatch；不能把 Agent 示例里的 structured_response 字段套用到这个返回值。

4.6 质量门禁：结构合法的题，仍可能被拒绝

validate_question_drafts 接收知识点、候选题、真实片段和请求约束，返回 accepted 与 rejected 两个列表。被拒绝的对象保留完整题目和第一项拒绝原因，便于界面复盘。它采用确定性规则，没有在这里调用第二个模型评审，也没有自动修复题目循环。

源码节选：请求范围与原文匹配

定位：apps/api/study_agent/question_generation/quality.py，第 33-51 行。

```python
    for question in questions:
        reason: str | None = None
        if question.question_type not in allowed_types:
            reason = "题型不在请求范围内"
        elif question.knowledge_point_key not in point_keys:
            reason = "知识点不存在"
        elif not difficulty_min <= question.difficulty <= difficulty_max:
            reason = "难度超出请求范围"
        elif any(chunk_id not in chunk_map for chunk_id in question.source_chunk_ids):
            reason = "引用了不存在的片段"
        elif len(question.source_chunk_ids) != len(question.source_quotes):
            reason = "片段与引文数量不一致"
        else:
            for chunk_id, quote in zip(
                question.source_chunk_ids, question.source_quotes, strict=True
            ):
                if _normalize(quote) not in _normalize(chunk_map[chunk_id].content):
                    reason = "引文无法在原文片段中匹配"
                    break
```

chunk_map 与 point_keys 在循环前从真实输入建立；循环按候选题顺序执行。

题型、知识点、难度和来源存在性按顺序检查；第一个失败原因优先记录。

zip(strict=True) 前先检查列表长度；引文匹配忽略空白和大小写。

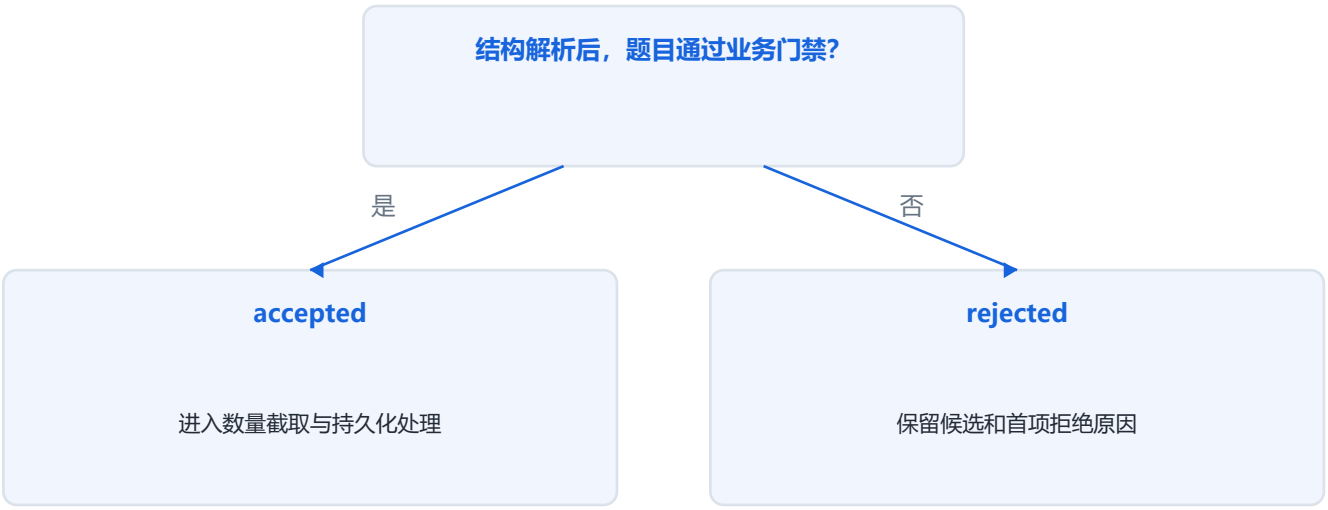


图 4-4 质量分类分支。任一业务规则失败即归入 rejected；没有自动修复循环。

检查	会拦截什么	不能据此证明什么
来源 ID	不存在的原文片段	答案一定被该片段支持
引文匹配	编造、改变事实的引文	题干和解析的推理都正确

检查	会拦截什么	不能据此证明什么
同批题干	忽略空白和大小写后的重复	语义相近但措辞不同的题目已去重
单选选项	规范化后文字重复	干扰项设计有区分度
填空答案	单个答案长度超过 100	所有合理同义答案都已列出

表 4-5 程序门禁的作用与边界。

例如候选题引用真实句子，却把正确答案写成“五次”，引文字符串仍可能匹配成功。校验的作用是缩小错误范围，语义充分性仍需要人工阅读“题干、答案、解析、引文”四者是否一致。题目生成后保留 draft 状态，就是为这种复核留出明确入口。

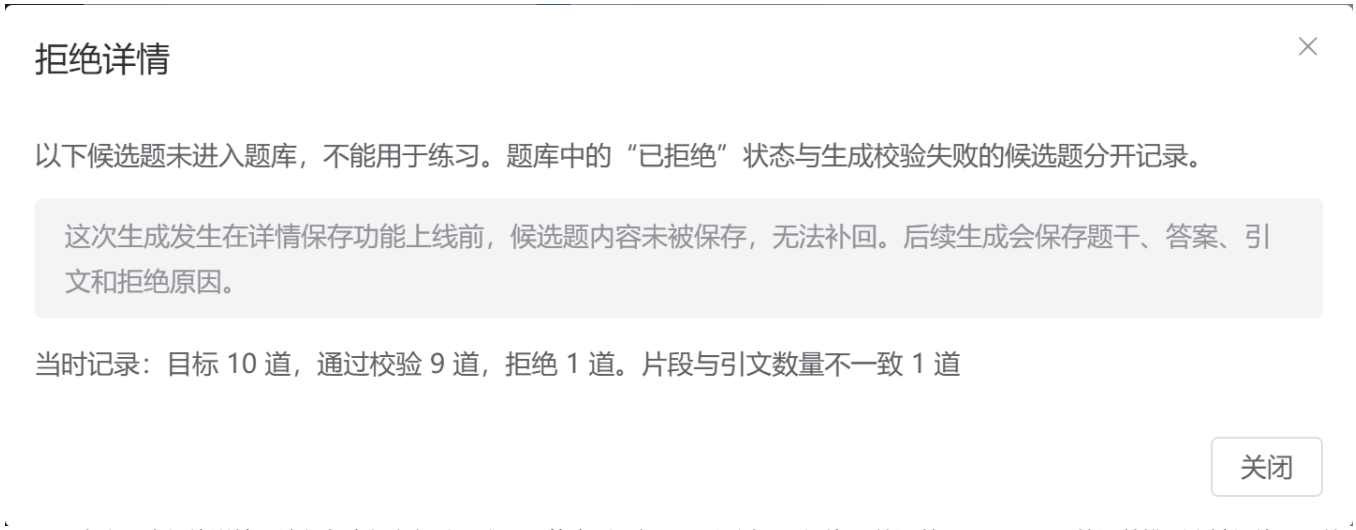


图 4-5 真实历史拒绝详情：该任务未保存候选正文，只能查看目标 10 / 通过 9 / 拒绝 1 的汇总及原因。不要从汇总推测出被拒绝题目的具体内容。

当前真实项目页面 / 现有业务数据 / 2026-10-07。历史候选正文未保存；展示已记录的拒绝汇总与原因

截图中的历史任务发生在候选正文保存功能上线之前，页面已明确提示无法补回。汇总拒绝原因能帮助定位门禁阶段，但不支持逐字审查那道候选题；逐题审查应使用实际保存的候选正文和引用。题库审核中的已拒绝状态与生成时未入库的拒绝候选也要区分。

4.7 保存与界面：为什么会出现 partial 和待审核?

自动通过的题目先按目标题数截取，再计算题型、规范化题干和答案的 SHA-256 内容哈希。任务查找知识库内相同哈希题目，复用有效题并更新来源；已经被删除的相同题会记为拒绝，不会偷偷重新启用。这里是精确内容去重，没有实现语义向量相似度去重。

阶段 / 状态	界面含义	核对位置
extracting_knowledge_points	正在抽取知识点；progress=10	是否已读到资料片段
generating_questions	正在生成候选；progress=45	结构化输出是否可解析
persisting	开始保存通过题；progress=75	来源、内容哈希与数据库关系

阶段 / 状态	界面含义	核对位置
completed	保存计数达到目标；progress=100	不等于人工审核全部通过
partial	有效保存计数少于目标；progress=100	查看拒绝原因或候选数量不足
draft / active	待审核题 / 可用于学习的启用题	在题库中复核后启用

表 4-6 进度数值是固定阶段标记，不是模型完成百分比预测。



图 4-6 真实资料详情页，资料为 api-server接口能力梳理.pdf。观察生成目标、通过数和拒绝数；这组界面记录用于解释作业状态。

当前真实项目页面 / 现有业务数据 / 2026-10-07。材料已就绪与生成目标10/通过9/拒绝1的实际记录



图 4-7 真实题目详情页。逐项对照题干、答案、解析和原文引文；它是人工复核内容充分性的阅读位置。

当前真实项目页面 / 现有业务数据 / 2026-10-07。题目、标准答案、解析与原文引文的真实审核界面

来源关系与题目保存之后，任务还会写入题库向量索引并再次检查题目是否在索引期间被删除。生成记录写明 execution_kind="chain"，有知识点和题目两个步骤摘要。历史数据库字段保留 agent_type 等名字，是兼容记录格式，不能据此把当前执行流程解释成 Agent。

计数的准确含义

generated_count 来自 saved_models 长度，其中可能包含复用的既有题目；不能一律称为“本次新增题数”。拒绝候选还可能包含超出目标的题，以及已删除的相同题。

4.8 完整离线实验：一次解析，两种质量结果

把下面代码保存为临时 Python 文件，在项目已准备的解释器中执行。FakeListChatModel 固定返回我们提供的 JSON，不读取 .env、不连接模型服务，也不写数据库。实验用真实 Pydantic 模型和质量函数验证同一事实：能解析为题目对象，与能通过引用校验，是两件事。

完整可运行实验：LCEL 解析与来源门禁

```python
from uuid import UUID
from langchain_core.language_models.fake_chat_models import FakeListChatModel
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate
from study_agent.domain.models import DocumentChunk
from study_agent.question_generation.schemas import (
    GeneratedQuestionDraft, KnowledgePointDraft, QuestionBatch,
)
from study_agent.question_generation.quality import validate_question_drafts

chunk = DocumentChunk(
    id=UUID(int=1), material_id=UUID(int=2), chunk_index=0,
    content="TCP 通过三次握手建立连接。", token_count=16,
    page_start=1, page_end=1, heading_path=["网络", "TCP"],
    content_hash="offline-demo", vector_id=None, embedding_model=None,
)
point = KnowledgePointDraft(
    canonical_key="tcp-handshake", name="TCP 三次握手",
    description="理解 TCP 建立连接的过程", importance=0.9,
    difficulty=2, source_chunk_ids=[chunk.id],
)
good = GeneratedQuestionDraft(
    knowledge_point_key=point.canonical_key, question_type="true_false",
    stem="TCP 通过三次握手建立连接。", correct_answers=["正确"],
    explanation="这一陈述与提供的原文一致。", difficulty=2,
    source_chunk_ids=[chunk.id], source_quotes=["TCP 通过三次握手建立连接"],
)
bad = good.model_copy(update={
    "stem": "TCP 建立连接需要五次握手。",
    "source_quotes": ["TCP 通过五次握手建立连接"],
})
parser = PydanticOutputParser(pydantic_object=QuestionBatch)
prompt = ChatPromptTemplate.from_messages([
    ("system", "只按原文出题。{format_instructions}"),
    ("human", "原文：{context}"),
]).partial(format_instructions=parser.get_format_instructions())
model = FakeListChatModel(
    responses=[QuestionBatch(items=[good, bad]).model_dump_json()]
)
batch = (prompt | model | parser).invoke({"context": chunk.content})
accepted, rejected = validate_question_drafts(
    knowledge_points=[point], questions=batch.items, chunks=[chunk],
    allowed_types={"true_false"}, difficulty_min=1, difficulty_max=3,
)
print("parsed =", len(batch.items))
print("accepted =", len(accepted), "rejected =", len(rejected))
print("reason =", rejected[0].reason)
assert len(accepted) == 1 and len(rejected) == 1
```

生产路径包含模型调用；这里使用脚本化聊天模型，固定复现输出形状。

第一道题引文来自真实输入，第二道题把“三次”改成“五次”。

model_copy 只用于制造测试候选；真正解析仍经过 QuestionBatch 和 Parser。

结果解释：实际输出 parsed = 2，accepted = 1 rejected = 1，拒绝原因为“引文无法在原文片段中匹配”。这不是模型准确率评测。



图 4-8 实验独立验证结构与来源两个层次。

继续运行预算实验。max_chars 故意设置为 10，第一段正文就已超过预算。你会看到第一段仍在上下文，第二段没有加入。这个极小参数用于暴露边界，不代表默认 24000 字符配置下的真实资料覆盖率；实际覆盖需要统计每份资料哪些章节进入了模型输入。

完整可运行实验：首片段预算边界

```
from dataclasses import replace
from uuid import UUID
from study_agent.domain.models import DocumentChunk
from study_agent.question_generation.tasks import build_generation_context

first = DocumentChunk(
    UUID(int=1), UUID(int=2), 0, "甲" * 50, 25, 1, 1,
    ["第一节"], "demo-1", None, None,
)
second = replace(first, id=UUID(int=3), chunk_index=1, content="后章内容")
context = build_generation_context([first, second], max_chars=10)
print("over_budget =", len(context) > 10)
print("first_included =", str(first.id) in context)
print("second_included =", str(second.id) in context)
assert len(context) > 10 and str(second.id) not in context
```

replace 创建独立片段对象，保持其他 metadata 不变。

三个布尔输出对应超额、首段加入和后段停止。

结果解释：实际输出 over_budget = True，first_included = True，second_included = False。

4.9 常见错误、练习与参考思路

最常见的阅读错误，是把提示词要求当成程序保证。提示词中要求“不编造”，程序仍要检查引文；提示词要求目标十道，最终也可能 partial。另一个错误是为了演示效果跳过 draft 审核直接启用所有题。学习这条链时，应先能解释一次候选被拒绝的原因，再讨论怎样提高生成数量。

- 练习一：把离线实验的引文改成空白差异，判断规范化匹配是否仍通过，再运行验证。
- 练习二：让两题题干完全相同，保留真实引文，观察“同批次题干重复”这一拒绝原因。
- 练习三：设计一道引用真实、答案却错误的题，说明当前自动门禁为何可能放行。
- 练习四：用三段材料估算预算，逐段写出 used；额外比较最终上下文包含的空行。

参考思路：第一题检查 _normalize 删除空白的行为；第二题检查 seen_stems 只记录被接受的题干；第三题提出人工支持性审核，而不是虚构已有语义审判器；第四题分别计算单段头部与正文、段间分隔符。练习要记录输入、观察输出和源码行，避免仅凭名称猜结论。

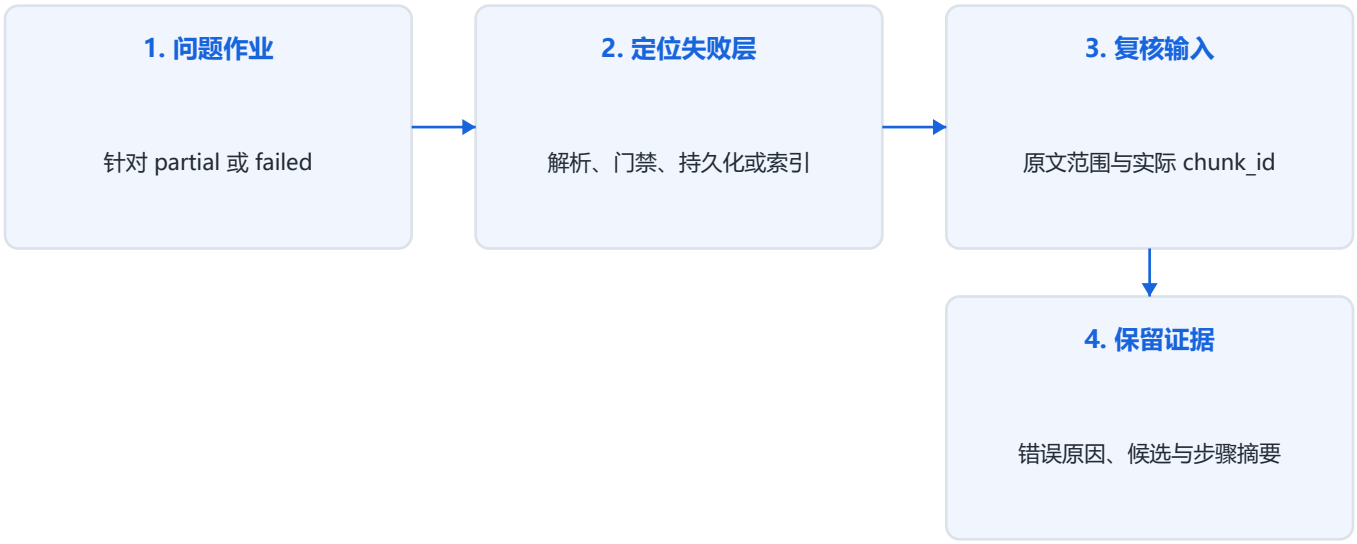


图 4-9 针对问题作业的排错路径。completed 的内容质量仍需另行审核。

学完以后应能说明

为什么本项目出题采用固定 Chain；为什么字符预算影响长资料；为什么结构化输出仍需要引用门禁；为什么待审核题不能直接代表高质量学习内容。

4.10 综合练习与参考思路

练习 1：题目生成为什么不是查询式向量 RAG?

参考思路：任务按资料片段顺序拼接上下文，没有查询问题与向量召回步骤。

练习 2：Parser 成功是否说明答案正确?

参考思路：只说明输出符合模型结构。请求范围、来源存在和引文还要校验；语义支持需要额外复核。

练习 3：24000 字符是否是严格上限?

参考思路：首段可以超额，段间两个换行未计入 used；这是软预算实现。

练习 4：completed 与 active 有何区别?

参考思路：前者是作业计数达到目标，后者是题目已启用可以进入学习。

练习 5：当前去重是否能识别同义改写?

参考思路：同批规范化题干与跨批内容哈希属于精确检查，没有语义相似度去重。

4.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
apps/api/study_agent/question_generation/chains.py	create_knowledge_point_chain / create_question_generation_chain	比较两条 LCEL 链的输入和返回类型
apps/api/study_agent/question_generation/tasks.py	build_generation_context / generate_questions	追踪材料预算、阶段推进、候选保存与结果计数
apps/api/study_agent/question_generation/schemas.py	GeneratedQuestionDraft / QuestionBatch	核对题型字段与跨字段校验
apps/api/study_agent/question_generation/quality.py	validate_question_drafts	解释 accepted 与 rejected 的条件
apps/api/study_agent/application/questions.py	create_generation_job / question_content_hash	核对作业前置条件与精确内容去重
apps/api/study_agent/config.py	Settings	核对 parser 默认策略与 24000 字符预算
apps/api/tests/test_question_generation.py	test_grounding_gate_rejects_fabricated_quote	阅读题型、结构与伪造引文的离线检查

下一章：下一章转入辅导 Agent，比较模型选择工具的动态循环与本章固定出题步骤。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。